

Leonor Rebola

Estudante nº 53663

Curso: Inteligência Artificial e Ciência de Dados

UC: Interação com Modelos de Larga Escala

Repositório GitHub

Covilhã, Junho de 2026

TP 2: Retail Vision Intelligence System

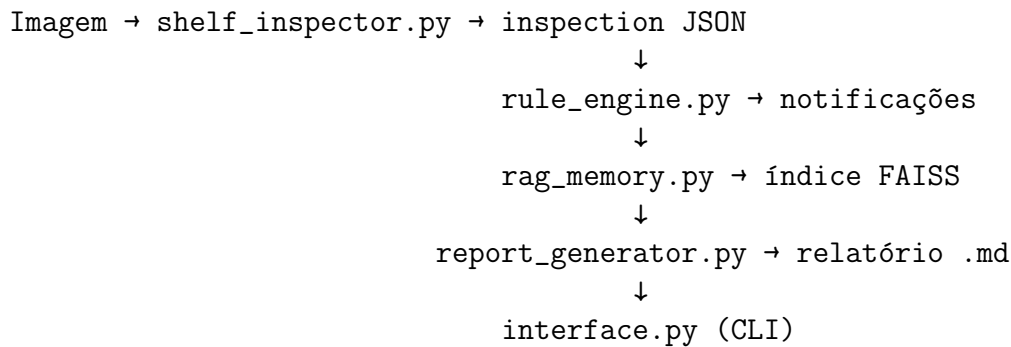
1 Introdução e arquitectura

O seguinte trabalho implementa um sistema de inspeção contínua de prateleiras de supermercado com memória histórica, capaz de analisar imagens com um modelo de linguagem multimodal, aprender as regras do gestor em linguagem natural e integrar a análise visual com dados de trajetória provenientes do primeiro trabalho prático.

O sistema é composto por cinco componentes:

- `shelf_inspector.py`: análise visual de imagens de prateleiras com Gemini 2.5 Flash, com três estratégias de prompting.
- `rule_engine.py`: conversão de regras em linguagem natural para configurações executáveis via LLM.
- `rag_memory.py`: indexação e recuperação semântica de inspeções históricas com FAISS e embeddings Gemini.
- `report_generator.py`: geração automática de relatórios de inspeção com contexto histórico
- `interface.py`: interface de linha de comandos que combina todos os componentes.

Em termos de arquitectura, a imagem é enviada para a API do Gemini que devolve um JSON estruturado com os problemas detectados. Este resultado é indexado no RAG, as regras do gestor são verificadas e um relatório é gerado automaticamente.



1.1 Decisões de design de alto nível

O modelo usado foi o `gemini-2.5-flash` em vez do `gemini-1.5-flash`, pois o modelo que foi sugerido não estava disponível na versão gratuita. Para a memória vectorial, foi utilizado FAISS com `IndexFlatIP` em vez do ChromaDB sugerido, por integração mais simples com a API já existente.

Os embeddings foram gerados com o modelo `gemini-embedding-001` (dimensão 3072).

2 Origem do dataset

O dataset deste trabalho é composto por **583 imagens**, provenientes das seguintes fontes:

- Fotos tiradas por mim em supermercados portugueses - 84 fotos;
- Imagens retiradas do dataset público SKU-110K (Goldman et al., 2019), acedido via HuggingFace - 497 imagens. O uso respeita a licença académica do dataset original.

Dentro do dataset, apenas 17 foram anotadas manualmente para constituir o ground truth de avaliação, cobrindo todas as sete zonas e os principais tipos de problema: `empty_shelf`, `wrong_product`, `misaligned` e `damaged`. As anotações incluem estado geral (*ok*, *warning*, *critical*), fill rate estimado, localização e severidade de cada problema.

2.1 Limitações e impacto na avaliação

O dataset apresenta três limitações principais:

- As 17 imagens anotadas não cobrem os tipos `label_missing` e `damaged` de forma representativa, o que pode enviesar as métricas de Issue Detection Rate para esses tipos.
- As imagens SKU-110K foram atribuídas a zonas de forma cíclica por ausência de metadados de localização, introduzindo ruído nas queries RAG que filtram por zona específica.
- As imagens próprias foram recolhidas num único dia, limitando a diversidade temporal do histórico indexado.

A anotação manual foi realizada com conhecimento direto do contexto de captura, já que se tratam das fotos que foram recolhidas em supermercados, o que garante maior fiabilidade do que a anotação remota.

3 Análise visual

3.1 Modelo e prompting

Para esta parte, foi usado o modelo **Gemini-2.5-flash** via API da Google e foram implementadas e comparadas três estratégias de prompting sobre as 17 imagens anteriormente anotadas com ground truth. As estratégias foram as seguintes:

- **Zero-Shot**: instrução direta pedindo o JSON estruturado, sem exemplos nem estrutura de raciocínio explícita.
- **Chain-of-Thought**: o modelo é guiado por quatro passos obrigatórios antes de produzir o documento JSON. Esses passos são: descrição geral, análise zona a zona, identificação de anomalias e classificação final.
- **Few-Shot**: dois exemplos de análises anteriores corretas são fornecidos como referência antes da imagem a analisar, sem consumir quota adicional por imagem extra.

3.2 Resultados quantitativos

Depois, foi corrido o código *shelf_inspector.py*, onde os resultados foram os seguintes:

Tabela 1: Comparação das três estratégias de prompting

Métrica	A	B	C
Issue Detection Rate	0.846	0.692	0.846
False Positive Rate	0.373	0.691	0.186
Severity Accuracy	0.708	0.611	0.625
JSON Parse Rate	1.000	1.000	1.000

De entre as estratégias, aquela que revelou melhores resultados foi a estratégia C onde, apesar de ter tido um Issue Detection Rate à estratégia A, teve um False Positive Rate significativamente inferior (0.186 vs 0.373).

Já a estratégia B, apesar de produzir análises mais detalhadas com raciocínio explícito por prateleira, gerou mais falsos positivos (0.691), penalizando a precisão.

Os resultados de JSON Parse Rate de 1.0 em todas as estratégias demonstra robustez do parsing implementado.

3.3 Análise de casos de falha

O caso de falha mais relevante ocorreu na IMG_9048 com a estratégia A. Esta imagem trata-se de uma prateria que contém sprays de ambientador Air Wick colocados entre produtos alimentares (no caso, cereais), constituindo uma violação grave de planograma classificada como *critical* no ground truth.

A estratégia A classificou a imagem como *ok* com fill rate de 0.95, não detectando o **wrong_product** e a estratégia B detectou o issue mas classificou-o como *warning*. Apenas a estratégia C identificou correctamente a gravidade da situação.

Um segundo caso de falha foi a tendência das estratégias A e B para responder em inglês em algumas imagens, apesar do prompt estar em português.

4 Rule Engine

4.1 Conversão de Regras

O Rule Engine converte regras em linguagem natural para um JSON estruturado e executável via LLM. O processo envolve três etapas, sendo elas a conversão, validação e execução. A conversão utiliza o modelo `gemini-2.5-flash` também com `temperature=0` para garantir reprodutibilidade.

Para isso, foram criadas 10 regras de teste, distribuídas entre regras claras e ambíguas. As regras estão divididas em:

- **5 regras claras:** especificam zona, threshold numérico, tipo de problema e nível de alerta de forma inequívoca. Exemplo: *"Na zona Z_S6, se o fill rate for inferior a 50% e houver pelo menos um empty_shelf de severidade high, gera alerta crítico imediato"*
- **5 regras ambíguas:** omitem informação necessária para execução determinística. Exemplo: *"Avisa-me quando a prateleira estiver vazia"* — não especifica threshold, zona nem urgência

4.2 Detecção de ambiguidades

Quando uma regra é ambígua, o sistema identifica os aspectos que não são claros, lista-os no campo `validation.ambiguities` e apresenta os pressupostos assumidos ao gestor. Por exemplo, para a regra **"Avisa-me quando a prateleira estiver vazia"**, o sistema identificou correctamente quatro ambiguidades - ausência de threshold numérico para "vazia", zona não especificada, urgência não definida e localização da prateleira não especificada.

4.3 Exemplos de conversão

Relativamente às regras claras, por exemplo, para a regra **"Na zona Z_S1, se o fill rate cair abaixo de 60%, avisa com nível crítico"** o sistema conseguiu converter de forma correcta a zona, o threshold, o nível de alerta e os campos `zone_filter: ["Z_S1"]`, `fill_rate_threshold: 0.6`, `alert_level: "critical"` e `is_valid: true`. Os únicos pressupostos assumidos foram o `severity_threshold`, que foi definido como *low* por omissão, e o `location_filter`, que foi definido como *any*, ambos razoáveis dado o contexto da regra.

Já para as regras ambíguas, por exemplo, **"Avisa-me quando a prateleira estiver vazia"** gerou `is_valid: false` com quatro ambiguidades listadas, sendo elas a ausência de threshold numérico para definir "vazia", zona não especificada, urgência não definida e localização da prateleira não especificada. O sistema guardou a regra, mas marcou-a como inválida, apresentando as ambiguidades e os pressupostos assumidos ao gestor.

As regras que são ambíguas não são executadas após as inspeções, pois aguardam clarificação do gestor.

4.4 Limitações

O modelo revelou tendência para classificar como ambíguas regras que tinham especificações. Por exemplo, a regra **"Na zona Z_S2, se houver empty_shelf na prateleira**

inferior, nível crítico” foi marcada como ambígua por ausência de threshold numérico para “vazia”, apesar de zona, localização e urgência estarem definidos, o que representa falsos positivos na detecção de ambiguidades.

Uma solução seria distinguir no prompt entre ambiguidades que impedem a execução e ambiguidades que conseguem ser resolvidas com pressupostos razoáveis. Isto confirma-se através das métricas de avaliação, onde o Rule Parse Rate foi de 0.8, onde uma regra acabou por não ser abrangida por causa da instabilidade da API, e a Ambiguity Detection de 1.0 nos casos genuinamente ambíguos.

5 RAG

5.1 Estratégia de Chunking

Para esta parte, foi adotada uma estratégia híbrida, onde cada inspeção é indexada como um chunk único cujo vector de embedding representa um sumário textual rico gerado pelo LLM, complementado com metadata estruturada (zona, data, fill rate, status, tipos de problema) guardada separadamente para filtragem pré-retrieval.

Esta alternativa acabou por ser escolhida por traduzir um equilíbrio entre as outras duas abordagens, pois indexar o record completo como chunk único perderia granularidade ao representar uma média de todo o conteúdo e indexar cada issue individualmente aumentaria o índice e dificultaria queries sobre o estado geral de uma zona.

5.2 Stack tecnológico

Em vez do ChromaDB, foi utilizado FAISS com `IndexFlatIP` (produto interno com vectores normalizados, equivalente a similaridade de cosseno) devido a incompatibilidades entre a biblioteca ChromaDB e a versão do PyTorch instalada no ambiente Windows e devido à API da HuggingFace estar inacessível na rede utilizada durante o desenvolvimento.

Os embeddings foram gerados via API do Gemini com o modelo `gemini-embedding-001` (dimensão 3072), já integrado na API existente, eliminando dependências adicionais.

5.3 Composição do índice

O índice foi construído com 77 inspeções únicas, sendo 34 inspeções das imagens fotografadas, que foram processadas com as estratégias B e C, e 43 inspeções do dataset SKU-110K, que foram processadas com a estratégia A.

A escolha das estratégias B e C para as imagens próprias justifica-se pelos melhores resultados obtidos na avaliação, acabando por produzir summaries mais ricos para indexação. Ambas as estratégias foram indexadas para a mesma imagem porque produzem summaries complementares, onde a estratégia B é mais detalhada com raciocínio explícito por prateleira, enquanto que a estratégia C é mais precisa e com menos ruído. A estratégia A ficou de fora do índice por ser redundante face às outras duas.

A seleção das inspeções a indexar foi automatizada pelo script `prepare_rag.py`, que filtra as inspeções por estratégia e origem da imagem, copiando automaticamente para a pasta `data/inspections_rag/` apenas as entradas relevantes.

Para as imagens SKU-110K, como não tinham ground truth associado, foram usadas apenas com a estratégia A por não fazer sentido gastar quota adicional da API.

Para além disto, as summaries foram enriquecidas com contexto de afluência histórica através do (`journeys.csv`) obtido através do primeiro Trabalho Prático, o que permitiu contextualizar inspeções com padrões de procura por zona e hora do dia.

5.4 Métricas de Retrieval

Nesta parte, foram obtidos os seguintes resultados:

Tabela 2: Métricas de avaliação do RAG

Métrica	Valor
Recall@3	0.6
Faithfulness	4.4/5
Answer Relevance	4.8/5

O Recall@3 de 0.6 indica que 3 das 5 queries de teste recuperaram pelo menos um documento relevante nos top-3 resultados. As duas queries que falharam não recuperaram os documentos esperados, possivelmente porque as summaries correspondentes não incluíam terminologia suficientemente próxima da query.

5.5 Limitações

A maior limitação trata-se do índice reduzido de 77 inspeções que, para um sistema de memória histórica em produção, é insuficiente para identificar padrões temporais robustos. Esta limitação deve-se a três fatores:

- A instabilidade recorrente do servidor Gemini durante o período de desenvolvimento, que causou falhas intermitentes nas chamadas à API (erro 503);
- As restrições da quota gratuita, que obrigaram a uma gestão cuidadosa das chamadas para garantir que os restantes componentes do sistema também pudessem ser desenvolvidos e testados;
- As restrições de tempo relativamente ao prazo de entrega, que não permitiram processar o dataset completo de 583 imagens.

Em produção, o índice cresceria automaticamente com o uso normal do sistema, onde cada inspeção realizada pela interface é indexada no RAG sem intervenção manual, eliminando a necessidade de processamento em batch.

6 Integração com o primeiro Trabalho Prático

Os dados de trajetória do Trabalho Prático 1 (`journeys.csv`) foram integrados na componente RAG para enriquecer o contexto dos summaries de inspeção. Para cada inspeção indexada, o sistema calcula a afluência histórica da zona correspondente à hora do dia da inspeção, comparando com a média histórica da mesma zona e hora.

6.1 Implementação

A integração foi implementada no módulo `journey_context.py`, que carrega o ficheiro e expõe a função `get_affluence_context(zone_id, hour)`, devolvendo uma descrição textual do nível de afluência. Esta descrição é incluída no prompt de geração de summary antes da chamada à API, enriquecendo o texto indexado com contexto operacional.

Por exemplo, uma inspeção que detecta prateleira vazia na zona Z_S1 às 15h com afluência 40% acima da média histórica sugere que a causa é falta de stock por procura elevada e não falha de reposição, tratando-se de uma distinção operacionalmente relevante para o gestor.

6.2 Limitações

Os dados de trajetória do primeiro trabalho cobrem o período de março de 2025, enquanto que as inspeções foram realizadas em junho de 2026. Devido a isso, não foi possível estabelecer uma correlação direta por data, acabando por a integração ser feita através das zonas e da hora do dia, assumindo que os padrões de afluência são relativamente estáveis ao longo do tempo para a mesma zona e hora independentemente da semana específica (por exemplo, a Z_S1 às 15h tem mais afluência do que às 9h). Esta é uma aproximação razoável para um sistema de retalho com horários regulares, mas introduz ruído quando existem, por exemplo, eventos especiais não capturados no histórico.

7 Conclusão

7.1 O que funcionou

O sistema cumpriu os objectivos principais do enunciado, sendo eles:

- Análise visual de prateleiras com três estratégias de prompting comparadas quantitativamente;
- Conversão de regras em linguagem natural com detecção de ambiguidades;
- Memória histórica com recuperação semântica e geração automática de relatórios.

A estratégia que revelou melhores resultados em termos de equilíbrio entre precisão e deteção foi a C, com um Issue Detection Rate de 0.846 e False Positive Rate de 0.186.

O JSON Parse Rate de 1.0 em todas as estratégias demonstra robustez do sistema de parsing e o LLM-as-Judge atribuiu 4.67/5 ao relatório gerado, com pontuação máxima na clareza e na rastreabilidade das citações.

Relativamente à integração com os dados das trajetória do primeiro trabalho, os mesmos enriqueceram as inspeções com contexto de afluência, apesar de não terem as mesmas datas em que as inspeções foram realizadas.

7.2 O que não funcionou

A principal dificuldade foi a instabilidade recorrente do servidor Gemini 2.5 Flash durante o desenvolvimento, com erros 503 (incapacidade de processar pedidos devido a sobrecarga) frequentes que obrigaram a gerir de forma cuidadosa a quota diária, o que acabou por limitar o volume de inspeções processadas e indexadas no RAG, resultando num índice

de apenas 77 inspeções.

A estratégia B, apesar de produzir análises mais detalhadas, gerou um False Positive Rate elevado (0.691), sugerindo que mais detalhe nem sempre se traduz em maior precisão.

Também, o Rule Engine revelou tendência para classificar como ambíguas regras parcialmente especificadas, gerando falsos positivos na detecção de ambiguidades.

7.3 O que faria de diferente

Com mais tempo disponível, implementaria as seguintes melhorias:

- **Dataset mais rico para o RAG:** processar o dataset completo de 583 imagens para o RAG, em vez das 77 inspeções actuais, permitindo identificar padrões temporais mais robustos;
- **Ground truth maior:** anotar manualmente pelo menos 50 imagens para métricas estatisticamente mais fiáveis;
- **Refinamento do Rule Engine:** distinguir no prompt entre ambiguidades bloqueantes e ambiguidades menores resolvíveis com pressupostos razoáveis;
- **Modelo com maior janela de contexto:** o limite de tokens do Gemini 2.5 Flash obrigou a dividir o relatório em dois grupos (um para as fotografias próprias e outro para as imagens SKU). Caso o acesso a um modelo com uma janela maior fosse possível, permitiria gerar um relatório unificado de todas as inspeções.

A Anexos

B Prompts

B.1 Estratégia A — Zero-Shot

Analisa esta imagem de uma prateleira de supermercado e produz uma análise estruturada em JSON.

O JSON deve seguir exatamente este schema:

```
{
  "overall_status": "ok|warning|critical",
  "issues": [...],
  "shelf_fill_rate": 0.0,
  "products_detected": [...],
  "model_reasoning": "..."
```

Regras importantes:

- affected_area_pct entre 0.0 e 1.0
- shelf_fill_rate entre 0.0 e 1.0
- Espaço no fim não é empty_shelf
- Se sem problemas, issues = [] e status = "ok"

Responde APENAS com o JSON. Sem texto adicional.

B.2 Estratégia B — Chain-of-Thought

Analisa esta imagem seguindo 4 passos obrigatórios:

PASSO 1 | DESCRIÇÃO GERAL

Descreve quantas prateleiras, que produtos, contexto geral.

PASSO 2 | ANÁLISE ZONA A ZONA

Para cada prateleira: está cheia? produtos alinhados?
produtos tombados? etiquetas visíveis?

PASSO 3 | IDENTIFICAÇÃO DE ANOMALIAS

Lista anomalias com localização e severidade.

PASSO 4 | CLASSIFICAÇÃO FINAL

Produz o JSON com base nos passos anteriores.

Escreve os 4 passos e termina com o JSON.

B.3 Estratégia C — Few-Shot

Analisa esta imagem de uma prateleira de supermercado.

EXEMPLO 1 | Prateleira normal:

```
{"overall_status": "ok", "issues": [],  
  "shelf_fill_rate": 0.95, ...}
```

EXEMPLO 2 | Prateleira com problemas:

```
{"overall_status": "critical",  
  "issues": [{"type": "empty_shelf", ...}], ...}
```

Agora analisa a imagem submetida seguindo o mesmo padrão.
Responde APENAS com o JSON.

B.4 Conversão de Regras — Rule Engine

És um sistema de conversão de regras de inspeção de prateleiras de supermercado.

O gestor de loja vai escrever uma regra em linguagem natural.
A tua tarefa é converter essa regra para JSON estruturado.

Schema obrigatório:

```
{  
  "rule_id": "<gerado externamente>",  
  "natural_language": "<texto original>",  
  "description": "reformulação clara em português formal",  
  "conditions": {  
    "zone_filter": [],  
    "time_filter": null,  
    "issue_types": [],  
    "severity_threshold": "low",  
    "fill_rate_threshold": null,  
    "location_filter": "any"  
  },  
  "action": {  
    "alert_level": "info|warning|critical",  
    "notification_message": "template da mensagem"  
  },  
  "validation": {  
    "is_valid": true,  
    "ambiguities": [],  
    "assumptions": []  
  }  
}
```

DETEÇÃO DE AMBIGUIDADES | marca is_valid=false quando:

- "vazia" sem threshold numérico concreto
- zona não especificada
- urgência não definida

- termos vagos sem valor numérico

Responde APENAS com o JSON.

B.5 Geração de Summary — RAG

És um sistema de indexação de inspeções de prateleiras de supermercado.

Dado o seguinte resultado de inspeção em JSON, gera um summary rico em termos semanticamente relevantes para recuperação futura.

O summary deve incluir obrigatoriamente:

- Zona inspecionada
- Data e hora
- Fill rate
- Tipos de produtos detetados
- Problemas encontrados com localização e severidade
- Estado geral

Exemplo de bom summary:

"prateleira inferior da zona Z_S3 com fill rate de 72%, produto de limpeza fora de posição na secção central, terça-feira 15h, estado warning"

Exemplo de mau summary:

"prateleira com problemas"

Responde APENAS com o texto do summary.

B.6 Resposta RAG

És um assistente de análise de inspeções de prateleiras de supermercado.

Com base nas inspeções históricas recuperadas abaixo, responde à query do gestor.

Regras:

- Cita sempre as inspeções pelo inspection_id e data
- Se os dados não forem suficientes, diz claramente
- Responde em português
- Sê direto e objetivo

Inspeções recuperadas: {context}

Query: {query}

B.7 Geração de Relatório

És um sistema de geração de relatórios de inspeção de prateleiras de supermercado.

Gera um relatório em Markdown com as seguintes secções:

1. Sumário Executivo

Máximo 150 palavras. Estado geral, zonas inspecionadas, issues críticos, warnings. Linguagem direta e acionável.

2. Problemas por Zona

Para cada zona com problemas: lista de problemas, severidade, fill rate, comparação com histórico.

3. Regras Disparadas

Regras ativadas, com que dados e que ação foi gerada.

4. Contexto Histórico Relevante

Padrões passados com referências explícitas (inspection_id, data).

5. Recomendações

Máximo 5 ações concretas ordenadas por urgência.

Dados da sessão: {session_data}

Contexto histórico: {historical_context}

Notificações: {notifications}

Gera o relatório completo. Sê direto e objetivo.

B.8 LLM-as-Judge

És um avaliador de sistemas de inspeção de prateleiras de supermercado.

Avalia o seguinte output do sistema com base no critério fornecido.

Critério de avaliação: {criterion}

Output do sistema:

{prediction_text}

Responde APENAS com este formato JSON:

```
{"score": <0 a 5>,  
  "justificacao": "<frase curta explicando o score>"}
```

Sem texto adicional, sem markdown.

C Exemplos de Inspection Records

C.1 Inspeção com estado *ok* — IMG_8990

```
{
  "inspection_id": "INS_20260610_164954_C96A4A",
  "timestamp": "2026-06-10T16:49:54Z",
  "image_path": "data/images/IMG_8990.jpg",
  "zone_id": "Z_S4",
  "strategy": "B",
  "overall_status": "ok",
  "shelf_fill_rate": 0.98,
  "products_detected": ["pensos higiénicos", "tampões",
                        "champô", "fraldas"],
  "issues": [
    {
      "issue_id": "ISS_001",
      "type": "misaligned",
      "location": "Prateleira 2, secção AUSONIA",
      "severity": "low",
      "confidence": 0.95,
      "affected_area_pct": 0.01
    }
  ],
  "summary": "Zona Z_S4 com fill rate de 98%, produtos de
  higiene feminina. Desalinhamentos de baixa severidade
  distribuídos pelas prateleiras. Estado ok."
}
```

C.2 Inspeção com estado *warning* — IMG_9058

```
{
  "inspection_id": "INS_20260610_160549_513070",
  "timestamp": "2026-06-10T16:05:49Z",
  "image_path": "data/images/IMG_9058.jpg",
  "zone_id": "Z_S6",
  "strategy": "A",
  "overall_status": "warning",
  "shelf_fill_rate": 0.77,
  "products_detected": ["Gin Mix", "Wine",
                        "Port Wine", "Gin accessories"],
  "issues": [
    {
      "issue_id": "ISS_001",
      "type": "empty_shelf",
      "location": "Fourth shelf, left side",
      "severity": "medium",
      "confidence": 0.95,
      "affected_area_pct": 0.5
    }
  ]
}
```

```

    },
    {
      "issue_id": "ISS_002",
      "type": "empty_shelf",
      "location": "Fifth shelf",
      "severity": "high",
      "confidence": 0.95,
      "affected_area_pct": 0.9
    }
  ]
}

```

D Exemplos de Regras Convertidas

D.1 Regra Clara — RULE_6B3F9E

```

{
  "rule_id": "RULE_6B3F9E",
  "natural_language": "Na zona Z_S6, se o fill rate for inferior a 50% e houver pelo menos um empty_shelf de severidade high, gera alerta crítico imediato",
  "description": "Gerar um alerta crítico imediato se, na zona Z_S6, o fill rate for inferior a 50% e for detetada uma prateleira vazia com severidade alta.",
  "conditions": {
    "zone_filter": ["Z_S6"],
    "issue_types": ["empty_shelf"],
    "severity_threshold": "high",
    "fill_rate_threshold": 0.5,
    "location_filter": "any"
  },
  "action": {
    "alert_level": "critical",
    "notification_message": "Alerta CRÍTICO: Zona {zone_id} com fill rate de {fill_rate}."
  },
  "validation": {
    "is_valid": true,
    "ambiguities": [],
    "assumptions": []
  }
}

```

D.2 Regra Ambígua — RULE_2DAE98

```

{
  "rule_id": "RULE_2DAE98",
  "natural_language": "Avisa-me quando a prateleira

```

```

    estiver vazia",
"conditions": {
    "zone_filter": [],
    "issue_types": ["empty_shelf"],
    "severity_threshold": "low",
    "fill_rate_threshold": null,
    "location_filter": "any"
},
"action": {
    "alert_level": "info"
},
"validation": {
    "is_valid": false,
    "ambiguities": [
        "Threshold numérico não especificado",
        "Zonas não especificadas",
        "Nível de urgência não definido",
        "Localização da prateleira não especificada"
    ],
    "assumptions": [
        "Interpretado como empty_shelf",
        "Aplicado a todas as zonas",
        "Alert_level definido como info por omissão"
    ]
}
}

```

E Relatório de inspeção

O relatório completo encontra-se no ficheiro `data/reports/report_20260611_091831.md` do repositório. Apresenta-se abaixo um excerto com as secções principais:

```

# Relatório de Inspeção | 2026-06-11 09:12 UTC

## 1. Sumário Executivo
A inspeção de 11 de junho de 2026 revelou um estado geral misto,
com 1 inspeção crítica e 9 warnings em 7 zonas inspecionadas.
O problema mais grave é a rutura de stock na zona Z_S2.

## 2. Problemas por Zona (excerto)
### Zona Z_S2
- Estado: Critical | Fill Rate: 67%
- Empty Shelf (High): Prateleira 4 completamente vazia
- Empty Shelf (High): Prateleira 5 completamente vazia

## 3. Regras Disparadas
- [RULE_878F3E] warning | wrong_product detetado na Z_S4
- [RULE_7C0C5B] warning | fill rate Z_S3 caiu para 0.67

## 4. Contexto Histórico
- [INS_20260610_174851_8F62EB] Z_S6 ok em 2026-06-10 17h48

```

5. Recomendações

1. Reabastecer imediatamente Zona Z_S2
2. Corrigir produtos fora do lugar nas Zonas Z_S4, Z_S3, Z_S5
3. Reabastecer prateleiras vazias de severidade média
4. Realinhar produtos em todas as zonas
5. Verificar visibilidade de etiquetas